Machines, Preferences, and World Models

An introduction to prediction, preference, and representation

Kyle Wright

working draft — October 15, 2026

© 2026 Kyle Wright. All rights reserved.

Prologue

These notes have arisen as a result of my personal transition to Artificial Intelligence (AI) safety and alignment research. A necessary first step is familiarising myself with the mathematical framework used to describe AI systems. I have a mathematical physics (string theory) background and have found it difficult to source notes pitched at the appropriate level and/or perspective for the questions I am interested in.

The field of AI safety and alignment aims to ensure increasingly capable AI systems reliably do what their designers and users intend, without causing serious harm. A central problem is alignment: the objectives AI systems actually learn can differ from the ones they were trained against, and the gap may not be easily detectable or correctable (Hubinger et al., 2019; Langosco et al., 2022; Shah et al., 2022). Solving it involves deciding what values the system should have, specifying them to the AI model, and ensuring it follows them.

One route to understanding the alignment problem, and the one taken here, is to ask how, and to what extent, an AI stores concepts. For current Large Language Models (LLMs) there is a somewhat trivial answer: the model is a function dependent on billions of parameters, called weights and biases[1] — so all information, and any notion of 'concepts', is stored in the parameters. However, the parameters are not human interpretable. It is natural to ask if AI behaviour could be described in a way that is understandable by humans. In physics we are not content with equations that describe the universe; we want intuition. We aim to quantitatively understand simple systems, with the hope the same qualitative behaviour persists in more complicated systems. Fundamentally there is no reason that this should be possible, but in physics we have found that it is the case in practice. There is empirical evidence that the same is true for LLMs, coming from mechanistic interpretability (Elhage et al., 2022; Olsson et al., 2022; Templeton et al., 2024), computational mechanics (Shai et al., 2024), and the theory of deep learning (Hoogland et al., 2025; Kaplan et al., 2020; Roberts et al., 2022; Watanabe, 2009). One hopes some of the lessons will hold for other AI systems.

Physics often proceeds by proposing a theory, typically a Lagrangian or Hamiltonian, deriving its consequences, and checking them against experiment; confidence grows as the checks accumulate. One can imagine the reverse: start from few assumptions and learn a model directly from all available data. For the universe this has been impractical. For language it is, roughly, what large language models do, and it has worked remarkably well. Such a model is built with comparatively little structure imposed by hand, yet it must end up storing some model of its data. What that model is, and how it could be read out, is the question these notes take up.

The notes are written for readers with a similar background to myself: someone with a PhD in mathematical physics, comfortable with abstract algebra and categorical language. Those with a fondness for geometric formalisms will find more to like in later chapters. Mathematical

[1] Other discrete choices, such as activation functions and architecture, also matter.

maturity is assumed, and perhaps a physicist's habit of checking theory against experiment. No background in machine learning, decision theory, or economics is assumed; results from those fields are stated and, where short enough, proved.

Disclaimer

These are working notes, not a finished or peer-reviewed text. They may contain errors, including in the statements of results. Items marked TODO are unverified or incomplete, and references should be checked against the original sources before being relied on. Results attributed to other authors are stated in my own words and may differ from the original formulations.

Parts of these notes were drafted with the help of an LLM (Claude). I have reviewed and revised all of the text, and any errors are my own.

The views expressed are my own and do not represent those of any organisation I am or have been affiliated with.

Contents

Prologue 1

Introduction 4

I Machines and their representations 7

1 Machines and their realisations 8
1.1 Alphabets, strings, and machines . . . 8
1.2 Probabilistic predictors, stochastic maps, and random maps . . . 9
1.2.1 The distribution monad and its Kleisli category . . . 11
1.2.2 Sampling: from predictions to stochastic outputs . . . 13
1.3 Machines as state machines . . . 14
1.3.1 Deterministic state machines . . . 14
1.3.2 State machines for predictors, stochastic maps, and random maps . . . 16
1.4 State machines as coalgebras . . . 17
1.4.1 One step: state machines are coalgebras . . . 17
1.4.2 Strings as the initial algebra . . . 18
1.4.3 Behaviours and the final coalgebra . . . 19
1.4.4 The variants as coalgebras . . . 21
1.5 Minimal realisations . . . 22
1.5.1 The Myhill–Nerode theorem . . . 22
1.5.2 The categorical version . . . 23
1.5.3 Minimal realisations of the variants . . . 24

Introduction

A trained language model is a fixed function. It reads a string of tokens and returns an output without being told what the task was or how the output is used. The loss it was trained against is never seen by the model, and the softmax that turns its outputs into probabilities was put there by its designers. The outputs are interpreted as probabilities, or as beliefs, by whoever uses the model. Nevertheless, whatever the model has learned about the world must be stored somewhere in it, and its behaviour must be explained by what is stored. These notes ask what such a model must store to behave as it does, and what preferences, utilities, or objectives its behaviour reveals. They also ask how much of either can be recovered from behaviour alone.

The questions matter for AI safety. Whether a system's learned objectives match the ones it was trained against is not something that can be checked from its loss curve — it is a question about what the system represents. There is evidence that the question has concrete answers in toy models. A transformer trained only to predict the next token of text generated by a hidden Markov process comes to represent the belief states of that process: its current uncertainty about the hidden state of the generator (Shai et al., 2024). Nobody told it to compute beliefs. The structure was forced by the task, and it can be found by looking in its internal activations (the residual stream). Much of these notes is an attempt to say, in general, when structure of this kind is expected and how it might be represented.

The notes put an emphasis on identifying the underlying assumptions. Papers in the field rightly focus on the task at hand, so many standard assumptions go unstated. For a newcomer, it can be useful to keep track of all the assumptions, and sometimes challenge them. It can be instructive to work through possible generalisations, even if the final outcome is the standard assumed starting point.

The approach

The notes start from the barest description of an AI system, a machine: a function $f : A^* \to B$ from strings of tokens in a finite alphabet A to a set B of outputs. Everything else is added one assumption at a time, at the point where it is needed. Each assumption is recorded in a ledger (Appendix C) and every result lists the assumptions it uses.

Two kinds of question run through the notes. Structural questions ask what a machine can represent internally, and what it must represent in order to behave as it does. Normative questions ask what makes one machine better than another. Part I takes the first, Part II the second, and Part III brings them together.

Part I: machines and what they represent

Every machine can be computed by a state machine, which holds an internal state and updates it one token at a time. Among all such realisations there is a smallest. Its states are the classes of strings that no continuation can tell apart (Chapter 1). A machine that memorises its training strings needs more states the more it has seen; a machine that has found a rule generating its data needs only as many as the rule requires. This is the first form of the question that motivates Part I: when does a machine store a model of its data rather than the data itself?

Language models are usually introduced as maps $f : A^* \to \Delta(A)$, returning a probability distribution over the next token. Chapter 2 asks why. A machine that names a single token cannot generate text with the right variety, cannot combine its predictions along a string, and cannot say how sure it is. But many richer outputs (sets of possible tokens, rankings, weights) also combine along a string. The chapter identifies the further requirements, concerning generation, frequencies, and mixtures, that single out probability distributions. It then shows that a next-token predictor determines a stochastic process, a single distribution over infinite sequences. Chapter 3 asks the same kind of question of information: which measures of uncertainty are compatible with using predictions repeatedly, and what has to be assumed to arrive at Shannon entropy.

The remaining chapters of Part I return to what is stored. Every process can be generated by a hidden Markov model if the hidden state is allowed to be the whole past; the question is how small the hidden state can be. There is a canonical answer, the coarsest summary of the past that loses nothing about the future (Chapter 4), and a theory of when finite hidden Markov representations exist (Chapter 5). Chapter 6 asks when storing such a model is more efficient, in memory or in data, than memorising examples. Chapter 7 asks which representation a machine stores when several generate the same behaviour: representations of a given dimension are related by a change of basis, and only what is invariant under that freedom is fixed by behaviour.

Part II: what makes a machine good

The set B^{A^*} of all machines is vast. Part I asked which machines can be built; Part II asks which are better. Given a fixed input, to say that one output is better than another requires some notion of preference. The weakest structure is a relation comparing outputs pairwise. On its own it may not even allow a choice, since preferences can cycle. Chapter 8 adds assumptions one at a time and tracks the structure that emerges, which under countability conditions culminates in a single utility. Chapter 9 shows where linearity comes from, either from how repeated occurrences are evaluated or from how randomised outputs are treated, making the utility cardinal, so that it can be averaged.

When the value of an output depends on something the machine does not know, such as the next token, consistent preferences factor into a belief about the unknown and a utility over outcomes (Chapter 10). The belief recovered this way is the same kind of object Part I took as primitive, and coherence across strings recovers its updating by conditioning. A loss function is then not a primitive either: it is what a preference looks like after enough assumptions have been made. Training adds one more assumption, a rule for weighing strings against each other (Chapter 11). Proper scoring rules, log loss among them, are the losses that reward a predictor for reporting its belief (Chapter 12). This is how the externally imposed loss of training connects to the internal structure studied in Part I.

Part III: world models and decisions

A machine whose outputs are judged by their consequences far downstream must predict well beyond the next token, and it must choose among outputs by their long-term consequences. It needs a world model and a notion of preference of its own. In next-token prediction the loss is external; a machine that plans must evaluate outcomes itself, and nothing guarantees that its internal preferences match the external ones it was trained against. This is where the alignment question becomes precise. Part III is, for now, an outline of open questions: how a machine could represent a world model, a belief about where it is within it, and a utility over what may follow, and how much of this could be recovered from behaviour.

How to read the notes

Each part depends on the chapters before it, with two exceptions: Part II can be read after Chapter 1, referring back to Chapter 2 for beliefs and conditioning, and the appendices are for reference. Appendix A collects the measure theory the notes use, stated in the most general setting in which it holds, with the extra assumptions marked. Appendix B gives the general form of the structure behind repeated prediction. Category theory is used throughout to organise results, mostly in remarks, but the main line can be followed without it.

The notes are a working draft. Chapters 1–3 of Part I and Chapters 8–10 of Part II are written, and the appendices are complete. The remaining chapters are outlines, with the planned content of each section noted in grey.

Part I

Machines and their representations

Chapter 1

Machines and their realisations

To study AI without committing to a particular architecture, training method, or set of parameters, we first need a vocabulary general enough to describe any system that reads tokens and responds to them. This chapter provides that vocabulary. Nothing in it is specific to AI: the objects defined here are maps that take in strings of tokens from a fixed alphabet and return an output, together with the internal mechanisms that can produce such maps. A language model, a classifier, and an agent choosing actions are all instances of such maps. The rest of the notes build on these definitions.

Two points of view run through the chapter. The *extensional* view describes a system by its behaviour alone: a *machine* is a map from strings to outputs, which may be deterministic, may output a probability distribution, or may output a random element (§1.1–§1.2); two systems with the same behaviour are the same machine. The *operational* view describes how a system computes: a *state machine* holds an internal state, updates it one token at a time, and reads off an output from its current state (§1.3). A state machine *realises* a machine when the two agree on every string. Coalgebras connect the two views (§1.4). How small a realisation can be, a first glimpse of whether a system memorises its inputs or models them, is answered by the Myhill–Nerode theorem (§1.5).

1.1 Alphabets, strings, and machines

An *alphabet* A is a finite non-empty set, whose elements are called *tokens*. A *string* is a finite sequence of tokens, $c = a_1 a_2 \cdots a_n$ with $a_1, \ldots, a_n \in A$; the *empty string* ϵ has none. (In machine learning, a string given to a model is usually called a *context*.) The set of all strings is denoted A^*. A *machine* is a function $f : A^* \to B$ from strings over A to a set B of outputs. In this chapter B can be any non-empty set.

Throughout these notes alphabets are finite, and the set of possible outputs, or of outcomes over which a distribution is output, is finite unless stated otherwise. This matches the systems we ultimately want to describe, whose vocabularies and sets of possible outputs are finite. Strings are finite by definition, but A^* itself is countably infinite, and the space of machines has the cardinality of the continuum (Proposition 1.3).

From a categorical viewpoint, A^* is the free monoid on A, with the product given by concatenation and the empty string ϵ as unit; the functor $A \mapsto A^*$ is left adjoint to the forgetful functor $\mathsf{Mon} \to \mathsf{Set}$. A machine is a morphism $f : A^* \to B$ in Set, and the space of machines is the hom-set $\mathsf{Set}(A^*, B) = B^{A^*}$. The monoid structure of A^* plays no role in this definition; it

enters through state machines (§1.3), whose transitions are exactly actions of A^*.

Remark 1.1 (Inputs and outputs). The input and output spaces may differ. For a next-token predictor, B is the set of tokens or of distributions over them; for a classifier, a small set of labels; for an agent, a set of actions.

Example 1.2 (Six machines)**.** Let $A = \{a, b\}$ and $B = \{0, 1\}$ unless stated otherwise.

1. *Constant*: $f(c) = 0$ for every c.
2. *Last token*: $f(c) = 1$ iff c ends in a.
3. *Parity*: $f(c) = 1$ iff c contains an odd number of a tokens.
4. *Balance*: $f(c) = 1$ iff c contains as many a tokens as b tokens.
5. *Memoriser*: given a finite list of examples (c_i, y_i), with the c_i distinct, $f(c_i) = y_i$ and $f(c) = 0$ for every other c.
6. *A language model*: A is some token vocabulary, B is A or the distributions over it, and f is whatever the network computes.

Proposition 1.3. *If B is finite or countable and $|B| \geq 2$, the set B^{A^*} of all machines has the cardinality of the continuum. The machines that can be computed by some program form a countable subset.*

Proof. A^* consists of all finite strings over a finite alphabet. Listing them by length, and alphabetically within each length (shortlex order), shows that A^* is countably infinite. Thus $|B^{A^*}| \geq 2^{\aleph_0}$ since $|B| \geq 2$, and $|B^{A^*}| \leq \aleph_0^{\aleph_0} = 2^{\aleph_0}$ since B is countable. There are countably many programs, so countably many computable machines. □

For a probabilistic predictor the output set $\Delta(B)$ is uncountable, and the corresponding set of machines is larger still.

Almost every machine, in the sense of cardinality, is an arbitrary table with no finite description. Any useful theory must therefore single out a tiny subset. There are two very different ways to do so, and it is worth keeping them apart.

- **Structural restrictions** say which machines can be *built*: finitely many states, computability, a fixed network architecture, a bound on memory. They concern how a machine is realised, and the Myhill–Nerode theorem of §1.5 is the prototype.
- **Normative restrictions** say which machines are *better*: on each string, some outputs are preferred to others. They concern what a machine is for.

The two interact. The best machine for a task may turn out to need only a few states, and a requirement to be small, or learnable from data, may push a machine towards a particular internal structure. Whether a machine memorises its inputs or models them is the version of this question studied in Chapter 6. These notes take the two kinds of restriction in turn: Part I, which this chapter begins, is structural, and Part II is normative, starting from the weakest possible notion of preference.

1.2 Probabilistic predictors, stochastic maps, and random maps

So far a machine has been deterministic: each string $c \in A^*$ is sent to a single output $f(c) \in B$. Real systems often behave differently. A large language model computes, from a string, a probability distribution over the next token. When it generates text it then draws a token at random from that distribution. This section separates three kinds of machine:

1. *probabilistic predictors*, which output a distribution;
2. *stochastic maps*, which output a random element drawn from a distribution, with fresh randomness each time;
3. *random maps*, which are deterministic machines chosen at random once, before any string is read.

For a set Y, write $\Delta(Y)$ for the set of *finitely supported probability distributions* on Y: functions $\ell : Y \to [0,1]$ that vanish outside a finite set and satisfy $\sum_y \ell(y) = 1$. When Y is finite, $\Delta(Y)$ is the simplex of dimension $|Y| - 1$, and its vertices are the point masses δ_y, where

$$\delta_y(y') = \begin{cases} 1 & \text{if } y' = y, \\ 0 & \text{if } y' \neq y. \end{cases}$$

A function $h : Y \to Z$ induces a pushforward map on distributions,

$$\Delta(h) : \Delta(Y) \to \Delta(Z), \qquad \Delta(h)(\ell)(z) = \sum_{y \in h^{-1}(z)} \ell(y),$$

where the sum is taken over all y such that $h(y) = z$. This makes Δ a functor $\mathsf{Set} \to \mathsf{Set}$. Finitely supported distributions cannot spread probability over infinitely many points; for that one needs countably additive probability measures and, on uncountable sets, a σ-algebra, and the Giry monad on measurable spaces in place of Δ (Giry, 1982, see also §A.7). Since the output sets in these notes are finite, Δ suffices for machines. The one exception is random maps, whose laws live on the infinite set B^{A^*}.

Example 1.4. Let $Y = \{1, 2, 3\}$ with $\ell = (0.2, 0.3, 0.5)$, and let $h : Y \to Z = \{\text{odd}, \text{even}\}$ send each number to its parity. Then

$$\Delta(h)(\ell)(\text{odd}) = \ell(1) + \ell(3) = 0.7, \qquad \Delta(h)(\ell)(\text{even}) = \ell(2) = 0.3.$$

Probabilistic predictors. A *probabilistic predictor* is a machine whose outputs are distributions,

$$f : A^* \longrightarrow \Delta(B) \qquad \text{a morphism in } \mathsf{Set}.$$

Given a string c, the output is the distribution $f(c)$; nothing random happens. The forward pass of a language model, followed by its softmax, is a probabilistic predictor.

Stochastic maps. A *stochastic map* is described by the same kind of data, a function $f : A^* \to \Delta(B)$, but it outputs an element of B chosen at random: given a string c it produces a random output Y_c sampled from $f(c)$, written $Y_c \sim f(c)$. Each use of the machine draws fresh randomness, independent of every other use. To mark this reading we write

$$f : A^* \rightsquigarrow B \qquad \text{a morphism in the Kleisli category } \mathsf{Kl}(\Delta),$$

defined in §1.2.1. A language model generating text is a stochastic map.

A probabilistic predictor outputs a point of the simplex, and it composes with other functions in the ordinary way. A stochastic map outputs a token, and when its output is fed into another stochastic map the randomness must be averaged over (§1.2.1). The two are related by sampling: a stochastic map is a probabilistic predictor followed by a draw from its prediction (Proposition 1.13).

Random maps. A *random map* is a deterministic machine chosen at random: a random element F of B^{A^*}, whose law ν is a probability measure on the set of all machines, with the product σ-algebra. When ν is finitely supported, $\nu \in \Delta(B^{A^*})$. Once F has been drawn, the machine is deterministic, and its output for string c is $F(c)$. A language model run with a fixed random seed is of this kind: the seed is drawn once, and from then on the behaviour is a deterministic function of the string.

A random map determines a stochastic map, by forgetting everything except the law of the output on each string separately: $f(c)$ is the pushforward of ν along $\mathrm{ev}_c : B^{A^*} \to B$, which evaluates a machine at c. The converse fails. The law ν also fixes how outputs on *different* strings are correlated, and a stochastic map records no such correlations.

Example 1.5 (Same marginals, different machines)**.** Let $B = \{0, 1\}$ and fix two strings $c \neq c'$. The random map F_1 tosses one fair coin and gives its result on every string; F_2 tosses an independent fair coin for each string. On any single string, both give 0 or 1 with probability $\frac{1}{2}$, so both determine the same stochastic map, $f(c) = \frac{1}{2}\delta_0 + \frac{1}{2}\delta_1$. Their answers on c and c' are jointly distributed differently:

$(F(c), F(c'))$	$(0,0)$	$(1,1)$	$(0,1)$	$(1,0)$
F_1	$\frac{1}{2}$	$\frac{1}{2}$	0	0
F_2	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$

A stochastic map, which draws fresh randomness at every use, behaves like F_2; no stochastic map behaves like F_1. Averages of a score over strings cannot tell the two apart; the difference appears only in properties of several answers together. Note that the law of F_2 is not finitely supported, which is why random maps need general probability measures.

Remark 1.6 (Laws on infinitely many strings). For infinitely many strings, a law on B^{A^*} needs the product σ-algebra. Such a law is determined by its joint laws on finitely many strings at a time (Theorem A.5), and by Kolmogorov's extension theorem (Theorem A.37) every consistent family of such joint laws arises from one. A stochastic map fixes only the one-string marginals; the independent product of them is one random map with those marginals, but in general there are many.

Remark 1.7 (Multi-token and autoregressive machines). Outputs may themselves be strings, $B = A_{\mathrm{out}}^k$ or A_{out}^*. A common way to produce them is to iterate a next-token machine. If $A_{\mathrm{out}} \subseteq A$ and $g : A^* \to A_{\mathrm{out}}$, the *autoregressive extension* of g produces $y_1 = g(c)$, $y_2 = g(cy_1)$, $y_3 = g(cy_1y_2)$, and so on. With a stochastic g, each token is drawn from g applied to the string so far, so the second token depends on the random first one; how to compute its law is the subject of §1.2.1. A prediction about whole strings and predictions about single next tokens are different things; they agree only under a consistency condition, the chain rule, taken up in Chapter 2.

1.2.1 The distribution monad and its Kleisli category

Stochastic maps need their own notion of composition. Run a stochastic next-token map $g : A^* \rightsquigarrow A$ autoregressively from a string c (Remark 1.7). The first token a is drawn from $g(c)$, and the second is drawn from $g(ca)$, so the law of the second token is obtained by averaging over the first:

$$\Pr(y_2 = a' \mid c) = \sum_{a \in A} g(c)(a)\, g(ca)(a'). \tag{1.1}$$

Composing stochastic maps means averaging over the random intermediate. This composition is associative and has identities, so stochastic maps form a category, the Kleisli category of the distribution monad. This subsection constructs it.

The construction uses the notion of a monad. A *monad* on a category C is a functor $T : \mathsf{C} \to \mathsf{C}$ with natural transformations $\eta : \mathrm{id}_{\mathsf{C}} \Rightarrow T$ (the *unit*) and $\mu : T^2 \Rightarrow T$ (the *multiplication*) such that

$$\mu \circ T(\mu) = \mu \circ \mu_T, \qquad \mu \circ T(\eta) = \mathrm{id}_T = \mu \circ \eta_T.$$

Proposition 1.8 (The distribution monad)**.** *Δ is a monad on Set, with unit and multiplication*

$$\delta_Y : Y \to \Delta(Y), \quad y \mapsto \delta_y, \qquad \mu_Y : \Delta\Delta(Y) \to \Delta(Y), \quad \mu_Y(P)(y) = \sum_{\ell} P(\ell)\,\ell(y).$$

The unit makes a point into a point mass; the multiplication flattens a distribution over distributions into a single distribution, by averaging.

Proof. Naturality of δ and μ is a direct check. The unit laws say that flattening a point mass at ℓ, or a distribution over point masses, returns the original distribution. The associativity law says that a distribution over distributions over distributions can be flattened in either order with the same result, which is the statement that iterated finite sums can be reordered. □

Example 1.9 (The distribution monad on two tokens)**.** Let $Y = \{a, b\}$, and write a distribution on Y as the pair $(\ell(a), \ell(b))$.

- *Unit.* $\delta_Y(a) = \delta_a = (1, 0)$: the token a becomes the distribution that is certain of a.
- *Functor.* For $h : Y \to \{0, 1\}$ with $h(a) = 0$, $h(b) = 1$, the pushforward $\Delta(h)$ sends $(0.3, 0.7)$ to $0.3\,\delta_0 + 0.7\,\delta_1$: it relabels the outcomes. For the map $Y \to \{*\}$ to a one-point set, both outcomes are identified and their probabilities add: $(0.3, 0.7)$ is sent to $(0.3 + 0.7)\,\delta_* = \delta_*$.
- *Multiplication.* Take two distributions $\ell_1 = (0.2, 0.8)$ and $\ell_2 = (0.6, 0.4)$, and the distribution $P = \frac{1}{2}\delta_{\ell_1} + \frac{1}{2}\delta_{\ell_2}$ over them: an element of $\Delta\Delta(Y)$. Think of P as a two-stage experiment: choose one of two biased coins by a fair toss, then toss the chosen coin. Flattening gives the distribution of the outcome,

$$\mu_Y(P) = \tfrac{1}{2}(0.2, 0.8) + \tfrac{1}{2}(0.6, 0.4) = (0.4, 0.6).$$

- *Unit laws.* Flattening the point mass δ_{ℓ_1} returns ℓ_1, and flattening $0.2\,\delta_{\delta_a} + 0.8\,\delta_{\delta_b}$ returns $(0.2, 0.8)$ again.

The two-step law (1.1) is a flattening of exactly this kind: the first token chooses which distribution $g(ca)$ the second token is drawn from, and μ averages over that choice.

Definition 1.10 (Kleisli category)**.** The *Kleisli category* $\mathsf{Kl}(\Delta)$ has the same objects as Set. A morphism $f : Y \rightsquigarrow Z$ is a map $f : Y \to \Delta(Z)$. The identity on Y is the unit δ_Y, and the composite of $f : Y \rightsquigarrow Z$ and $g : Z \rightsquigarrow W$ is

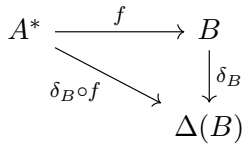
$$g \circ_{\mathrm{K}} f = \mu_W \circ \Delta(g) \circ f, \qquad (g \circ_{\mathrm{K}} f)(y)(w) = \sum_{z \in Z} f(y)(z)\, g(z)(w),$$

where $f(y)(z)$ is the probability of z given input y, also denoted $f(z|y)$, and similarly $g(z)(w) = g(w|z)$. As a diagram,

$$Y \xrightarrow{\ f\ } \Delta(Z) \xrightarrow{\ \Delta(g)\ } \Delta\Delta(W) \xrightarrow{\ \mu_W\ } \Delta(W).$$

The formula for composition is the Chapman–Kolmogorov equation (§A.5): to go from y to w through a random intermediate z, average over z, exactly as in (1.1). The monad laws are exactly what make this composition associative with identities δ. The same construction works for any monad (T, η, μ) on any category, with morphisms $Y \to TZ$, identities η_Y, and composition $\mu \circ Tg \circ f$; for the Giry monad it gives Markov kernels between measurable spaces.

Proposition 1.11 (Deterministic machines are stochastic maps)**.** *The assignment $f \mapsto \delta \circ f$, identity on objects, is a faithful functor $\mathsf{Set} \to \mathsf{Kl}(\Delta)$. A deterministic machine $f : A^* \to B$ thus becomes the stochastic map $\delta_B \circ f$ whose output on every string is certain:*



The stochastic maps in its image are exactly those whose outputs are all vertices of the simplex.

Proof. $(\delta \circ g) \circ_{\mathrm{K}} (\delta \circ f) = \mu \circ \Delta(\delta \circ g) \circ \delta \circ f = \mu \circ \delta \circ \delta \circ g \circ f = \delta \circ g \circ f$, using naturality of δ and the unit law. Faithfulness holds because δ is injective. □

1.2.2 Sampling: from predictions to stochastic outputs

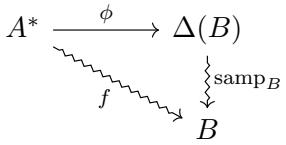
Definition 1.12 (Sampling)**.** The *sampling map* on B is the stochastic map

$$\mathrm{samp}_B : \Delta(B) \rightsquigarrow B, \qquad \mathrm{samp}_B = \mathrm{id}_{\Delta(B)} \text{ read as a function } \Delta(B) \to \Delta(B).$$

It takes a distribution as input and outputs a random element drawn from it.

Proposition 1.13 (A stochastic map is a prediction followed by a draw)**.** *Let $\phi : A^* \to \Delta(B)$ be a probabilistic predictor and $f : A^* \rightsquigarrow B$ the stochastic map with the same data. Then*

$$f = \mathrm{samp}_B \circ_{\mathrm{K}} (\delta_{\Delta(B)} \circ \phi),$$



In words: run the predictor, then draw from its prediction.

Proof. $\mu \circ \Delta(\mathrm{id}) \circ \delta \circ \phi = \mu \circ \delta \circ \phi = \phi$, by the unit law. □

Remark 1.14 (Composition is where the difference shows)*.* Feeding outputs back in exposes the difference between probabilistic predictors and stochastic maps. The autoregressive law (1.1) is a Kleisli composite: one averages over the sampled first token. A probabilistic predictor cannot be fed its own output in this way at all, since its output is a distribution and its input must be a string. Generating text from a predictor requires sampling, which turns it into a stochastic map (Proposition 1.13).

Kind	Type	Output on string c
deterministic machine	$f : A^* \to B$ in Set	$f(c) \in B$
probabilistic predictor	$f : A^* \to \Delta(B)$ in Set	the distribution $f(c)$
stochastic map	$f : A^* \rightsquigarrow B$ in $\mathsf{Kl}(\Delta)$	$Y_c \sim f(c)$, independent across uses
random map	$F \sim \nu$, ν a law on B^{A^*}	$F(c)$, correlated across strings

Deterministic machines are the special case of each of the other three: predictions that are point masses, stochastic maps whose outputs are certain, and random maps whose law is a point mass $\nu = \delta_f$. Which description is needed depends on what is asked of the machine. Anything computed one string at a time, such as an average score over strings, sees only the per-string distributions; properties involving several outputs together, such as consistency across queries, need the joint law of a random map (Example 1.5).

1.3 Machines as state machines

So far a machine has been a table, with one entry for each of the infinitely many strings in A^*. Every machine can also be read as a state machine: something that holds a state, updates it one token at a time, and emits an output from its current state. A table must store an entry for every string; a state machine stores only its states and its rules for moving between them, and it may be finite even though A^* is not. How few states suffice is the subject of §1.5.

1.3.1 Deterministic state machines

Definition 1.15 (State machine)**.** Fix an alphabet A and a set B. A (Moore-type) state machine is a quadruple (Q, q_0, τ, λ), where Q is a set of states, q_0 is a chosen initial state, $\tau : Q \times A \to Q$ a transition map and $\lambda : Q \to B$ an output map.

Remark 1.16 (Relation to finite automata). Definition 1.15 is the standard notion of a *Moore machine* (Moore, 1956), with two differences of emphasis: the set of states may be infinite, and the outputs may lie in any set. A *deterministic finite automaton* (Rabin and Scott, 1959) is the special case with Q finite and $B = \{0, 1\}$, the output map being the indicator of a set of accepting states. The machine it realises is then the indicator function of a set of strings, the *language* the automaton accepts, and the languages accepted by finite automata are the *regular* languages. A *Mealy machine* (Mealy, 1955) attaches outputs to transitions rather than to states. The two can be converted into each other, exactly up to the output on the empty string, which a Mealy machine does not produce; the conversion from Mealy to Moore may need more states, up to $|Q| \cdot |B|$. In this language, §1.5 characterises the machines that have a finite realisation, generalising the classical characterisation of regular languages.

The transition map tells the machine how to read a single token. Reading a whole string requires no extra structure: it is determined by reading the tokens one at a time, and there is only one way to do so.

Proposition 1.17 (Transitions extend uniquely to strings)**.** *There is a unique map $\tau : Q \times A^* \to Q$ that agrees with the transition map on single tokens and satisfies*

$$\tau(q, \epsilon) = q, \qquad \tau(q, ca) = \tau(\tau(q, c), a) \qquad (q \in Q,\ c \in A^*,\ a \in A).$$

Equivalently, $q \cdot c = \tau(q, c)$ is a right action of the monoid A^* on Q, and every right action of A^* on Q arises in this way from a unique one-step transition map.

Proof. The two equations define $\tau(q, c)$ by recursion on the length of c, which gives existence, and any two maps satisfying them agree on strings of every length, by induction, which gives uniqueness. For the second statement, a right action is a map $c \mapsto \rho_c \in \mathrm{End}(Q)$, $\rho_c(q) = q \cdot c$, with $\rho_\epsilon = \mathrm{id}$ and $\rho_{cc'} = \rho_{c'} \circ \rho_c$, that is, a monoid homomorphism $A^* \to \mathrm{End}(Q)^{\mathrm{op}}$ (the opposite monoid, because a right action composes from left to right). Since A^* is the free monoid on A, such homomorphisms correspond exactly to functions $A \to \mathrm{End}(Q)$, $a \mapsto \tau(\cdot, a)$, i.e. to one-step transition maps. □

Remark 1.18 (The extension is forced). So a state machine is specified by its one-step transitions alone, and the way it reads strings is a consequence. Some texts define the extension alongside the transition map, others define a state machine as an action of A^*; Proposition 1.17 says the two definitions agree. The uniqueness comes from the strings, not from the machine: A^* is built freely from A by appending one token at a time. Section 1.4.2 returns to this from the categorical side.

As in Proposition 1.17, write $q \cdot c = \tau(q, c)$ for the state reached from q by reading c, so that $q \cdot \epsilon = q$ and $q \cdot (cc') = (q \cdot c) \cdot c'$: by Proposition 1.17, the transition map makes Q a right A^*-set. The *behaviour* of a state q is the machine obtained by starting the state machine in q,

$$\mathrm{beh}_Q(q) : A^* \to B, \qquad \mathrm{beh}_Q(q)(w) = \lambda(q \cdot w).$$

A state machine *realises* the machine f if $f(c) = \lambda(q_0 \cdot c)$ for every string c. Writing $r_Q(c) = q_0 \cdot c$ for the *reachability map*, this says that the triangle

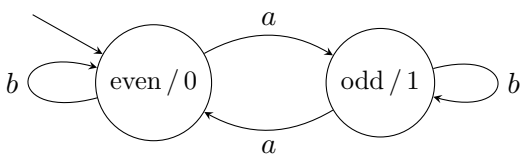
$$\begin{array}{ccc} A^* & \xrightarrow{r_Q} & Q \\ & \searrow^{f} & \downarrow{\scriptstyle \lambda} \\ & & B \end{array}$$

commutes: the machine is recovered by running the state machine from q_0 and reading off the output. Equivalently, a state machine realises f iff the behaviour of its initial state is f: $\mathrm{beh}_Q(q_0) = f$.

Definition 1.19 (Reachable and observable)**.** A state machine (Q, q_0, τ, λ) is *reachable* if every state is reached from the initial state by some string, i.e. the reachability map $r_Q : A^* \to Q$ is onto. It is *observable* if distinct states have distinct behaviours, i.e. $\mathrm{beh}_Q : Q \to B^{A^*}$ is one-to-one.

An unreachable state plays no part in realising f, and two states with the same behaviour cannot be told apart by any continuation. Both notions are used in §1.5.

Example 1.20 (Parity)**.** The parity machine of Example 1.2 is realised by a state machine with two states, $Q = \{\mathrm{even}, \mathrm{odd}\}$, initial state $q_0 = \mathrm{even}$, transitions that swap the state on reading a and keep it on reading b, and outputs $\lambda(\mathrm{even}) = 0$, $\lambda(\mathrm{odd}) = 1$:



Each state is labelled with its output. Reading a string from even ends in odd exactly when the string contains an odd number of as, so this state machine realises the parity machine. It is reachable, and it is observable: the two states give different outputs on the empty string.

Example 1.21 (The free realisation)**.** Every machine has a realisation: take the strings themselves as states, $Q = A^*$, with $q_0 = \epsilon$, $\tau(c, a) = ca$ and $\lambda = f$. The state after reading c is c itself, so this is the table of f read as a state machine. It is reachable, but it is usually far from observable: for the parity machine, any two strings with the same number of as modulo 2 have the same behaviour. Section 1.5 shows how to collapse it to the smallest possible realisation.

1.3.2 State machines for predictors, stochastic maps, and random maps

Each of the variants of §1.2 has a corresponding kind of state machine. They differ in where the randomness sits: in the output, in the initial state, or in the transitions.

Probabilistic predictors. A probabilistic predictor is a deterministic machine with output set $\Delta(B)$, so it is realised by an ordinary state machine (Definition 1.15) whose output map is $\lambda : Q \to \Delta(B)$. No new definition is needed.

Stochastic maps. A stochastic map can be realised by a state machine that moves deterministically on each input token and emits a random output from its current state.

Definition 1.22 (Stochastic state machine)**.** A *stochastic state machine* is (Q, q_0, τ, λ) as in Definition 1.15, except that the output map is a stochastic map $\lambda : Q \rightsquigarrow B$, i.e. a function $\lambda : Q \to \Delta(B)$. It *realises* the stochastic map $f : A^* \rightsquigarrow B$ if $\lambda(q_0 \cdot c) = f(c)$ for every string c.

Random maps. A random map is a deterministic machine chosen at random, so it is realised by a deterministic state machine whose initial state is chosen at random. In the following definition $\nu \in \Delta(B^{A^*})$ is the law of a random map, assumed finitely supported.

Definition 1.23 (State machine with random initial state)**.** A *state machine with random initial state* is $(Q, \nu_0, \tau, \lambda)$, where (Q, τ, λ) is as in Definition 1.15 and $\nu_0 \in \Delta(Q)$ is an *initial law*. The state machine *realises* ν if the behaviour of a random initial state has law ν:

$$\Delta(\mathrm{beh}_Q)(\nu_0) = \nu.$$

It is *reachable* if every state is of the form $q \cdot c$ with q in the support of ν_0.

Laws that are not finitely supported, such as that of F_2 in Example 1.5, need the Giry monad (§A.7) in place of Δ, both for ν and for ν_0.

Random transitions. Finally, the state itself may evolve at random.

Definition 1.24 (Probabilistic state machine)**.** A *probabilistic state machine* is (Q, q_0, τ, λ) with a set of states Q, an initial state q_0, a stochastic transition map $\tau : Q \times A \rightsquigarrow Q$ and a stochastic output map $\lambda : Q \rightsquigarrow B$. Writing $\tau_a = \tau(\cdot, a) : Q \rightsquigarrow Q$, the state after reading $c = a_1 \cdots a_n$ is random, with law

$$\tau_{a_n} \circ_{\mathrm{K}} \cdots \circ_{\mathrm{K}} \tau_{a_1} \circ_{\mathrm{K}} \delta_{q_0} \;\in\; \Delta(Q),$$

and the output after c has law $\lambda \circ_{\mathrm{K}} \tau_{a_n} \circ_{\mathrm{K}} \cdots \circ_{\mathrm{K}} \tau_{a_1} \circ_{\mathrm{K}} \delta_{q_0} \in \Delta(B)$ (here δ_{q_0} is read as a stochastic map $1 \rightsquigarrow Q$). The machine *realises* the stochastic map $f : A^* \rightsquigarrow B$ if this law is $f(c)$ for every string c.

Remark 1.25 (Input-driven hidden Markov models). A probabilistic state machine is a hidden Markov model driven by the input: the state is hidden, evolves at random, and is observed only through the outputs. The state after a string is not a function of the string. Within one run the hidden state persists, so outputs on successive strings of the same run are correlated; realisation as defined above concerns only the law of the output on each string, as befits a stochastic map.

State machine	Initial state	Transitions	Output	Realises
state machine	fixed	deterministic	deterministic	machine
— with $\lambda : Q \to \Delta(B)$	fixed	deterministic	a distribution	probabilistic predictor
stochastic state machine	fixed	deterministic	random	stochastic map
random initial state	random	deterministic	deterministic	random map
probabilistic state machine	fixed	random	random	stochastic map

1.4 State machines as coalgebras

State machines can be phrased categorically in terms of coalgebras. This section re-describes the definitions of §1.3 in that language. The description has three parts: one step of a state machine is a coalgebra (§1.4.1); strings, and the extension of τ to them, come from an initial algebra (§1.4.2); and behaviours, and machines themselves, come from a final coalgebra (§1.4.3).

1.4.1 One step: state machines are coalgebras

Throughout this section the alphabet A and the output set B are fixed, and $F : \mathsf{Set} \to \mathsf{Set}$ is the functor

$$F(Q) = B \times Q^A, \qquad F(h) = \mathrm{id}_B \times h^A,$$

where $h^A : Q^A \to Q'^A$ composes a function $A \to Q$ with h. An *F-coalgebra* is a set Q with a map $\alpha : Q \to F(Q)$. Coalgebras for different functors are different kinds of object, and morphisms are defined only between coalgebras for the *same* functor: a morphism of F-coalgebras $h : (Q, \alpha) \to (Q', \alpha')$ is a map $h : Q \to Q'$ making

$$\begin{array}{ccc} Q & \xrightarrow{\;h\;} & Q' \\ {\scriptstyle\alpha}\downarrow & & \downarrow{\scriptstyle\alpha'} \\ F(Q) & \xrightarrow[F(h)]{} & F(Q') \end{array}$$

commute: first step and then map, or map and then step. Unless another functor is named, as in §1.4.4, "coalgebra" in this section means F-coalgebra for this fixed F.

Proposition 1.26 (State machines and coalgebras are the same thing)**.** *A state machine without initial state, (Q, τ, λ), is exactly an F-coalgebra, via*

$$\alpha(q) = \big(\lambda(q),\ a \mapsto \tau(q, a)\big),$$

and a map $h : Q \to Q'$ is a coalgebra morphism iff it preserves outputs and transitions: $\lambda'(h(q)) = \lambda(q)$ and $h(\tau(q, a)) = \tau'(h(q), a)$. So state machines and F-coalgebras form isomorphic categories, $\mathsf{Coalg}(F)$.

Proof. A map $Q \to B \times Q^A$ is a pair of maps $Q \to B$ and $Q \to Q^A$, and the second is τ curried.[1] The morphism condition, read componentwise, is the stated pair of equations. □

Remark 1.27 (The coalgebra records one step). The coalgebra structure α records the transition $\tau(q, a)$ for single tokens $a \in A$ only, exactly as Definition 1.15 does. The extension of τ to strings is not part of it; it is recovered by iterating α one token at a time, as §1.4.2 shows.

A state machine *with* an initial state is a *pointed* coalgebra (Q, α, q_0), and morphisms of pointed coalgebras also preserve the point.

1.4.2 Strings as the initial algebra

The strings carry the dual kind of structure: an algebra rather than a coalgebra. Here $1 = \{*\}$ is a one-point set and $+$ is disjoint union, and we use the functor

$$G(X) = 1 + X \times A, \qquad G(h) = \mathrm{id}_1 + h \times \mathrm{id}_A.$$

A map out of a disjoint union is a pair of maps, one on each summand, and we write $[u, v] : 1 + X \times A \to X$ for the map that is u on 1 and v on $X \times A$. Since a map $1 \to X$ is just the choice of a point of X, a map $[x_0, v]$ amounts to a point $x_0 \in X$ together with a map $v : X \times A \to X$.

Definition 1.28 (G-algebras)**.** A *G-algebra* is a set X with a map $\beta : G(X) \to X$, i.e. a point $x_0 \in X$ and a map $v : X \times A \to X$, so that $\beta = [x_0, v]$. A morphism of G-algebras $h : (X, \beta) \to (X', \beta')$ is a map $h : X \to X'$ with $h \circ \beta = \beta' \circ G(h)$; on the two summands this says $h(x_0) = x_0'$ and $h(v(x, a)) = v'(h(x), a)$.

Two G-algebras matter here. A pointed state machine (Q, q_0, τ, λ) gives the G-algebra $[q_0, \tau] : 1 + Q \times A \to Q$, which sends $*$ to the initial state q_0 and (q, a) to $\tau(q, a)$: it records where the machine starts and how it reads one token. And the strings give the G-algebra $[\epsilon, \mathrm{append}] : 1 + A^* \times A \to A^*$, which sends $*$ to the empty string and (c, a) to ca. So a pointed state machine is simultaneously a G-algebra and an F-coalgebra on the same set, sharing τ.

Proposition 1.29 (Strings form the initial G-algebra)**.** *For every G-algebra $(X, [x_0, v])$ there is exactly one morphism of G-algebras $(A^*, [\epsilon, \mathrm{append}]) \to (X, [x_0, v])$. It is the map r defined by $r(\epsilon) = x_0$ and $r(ca) = v(r(c), a)$.*

Proof. A morphism r must satisfy the two equations on the summands: $r(\epsilon) = x_0$ and $r(ca) = v(r(c), a)$. Every string is *either* empty *or* of the form ca for a unique shorter string c and token a, so these equations define r on strings of each length in turn, by recursion on the length. This gives existence, and since any morphism must satisfy them, uniqueness. The decisive property is the unique decomposition of strings: the map $[\epsilon, \mathrm{append}] : 1 + A^* \times A \to A^*$ is a bijection. □

Remark 1.30. An initial object of a category is unique up to unique isomorphism, so the proposition characterises A^*, with its empty string and appending, up to relabelling. The bijectivity of $[\epsilon, \mathrm{append}]$ used in the proof is no accident: the structure map of any initial algebra is an isomorphism (Lambek's lemma (Lambek, 1968)). Initiality is the abstract form of *definition by recursion on strings*: to define a function on all strings it suffices to say where the empty string goes and how to take one more step.

[1] *Currying* turns a function of two arguments into a function of one argument whose values are functions: a map $g : X \times Y \to W$ corresponds to the map $X \to W^Y$ sending x to the function $y \mapsto g(x, y)$. This correspondence is a bijection between $W^{X \times Y}$ and $(W^Y)^X$. Here $\tau : Q \times A \to Q$ corresponds to the map $Q \to Q^A$ sending a state to its table of transitions.

For a pointed state machine, the unique morphism from the strings is the reachability map r_Q of §1.3.1: initiality of A^* says that r_Q is the unique map making

$$\begin{array}{ccc} 1 + A^* \times A & \xrightarrow{\mathrm{id}+r_Q\times\mathrm{id}} & 1 + Q \times A \\ {\scriptstyle [\epsilon,\ \mathrm{append}]}\downarrow & & \downarrow{\scriptstyle [q_0,\ \tau]} \\ A^* & \xrightarrow[r_Q]{} & Q \end{array}$$

commute: it sends the empty string to the initial state and commutes with reading one more token.

Remark 1.31 (Reframing the extension of τ and the reachability map). This recovers two items of §1.3.1. First, the extension of τ to strings: the two equations $\tau(q_0, \epsilon) = q_0$ and $\tau(q_0, ca) = \tau(\tau(q_0, c), a)$ of Proposition 1.17 are exactly the two components of the square above (its value on the summand 1 and on the summand $A^* \times A$). Initiality says these equations have exactly one solution, which is the uniqueness part of Proposition 1.17, from the initial state; replacing q_0 by any state q gives the extension from q. The two proofs of uniqueness, by initiality here and by freeness of the monoid A^* in Proposition 1.17, are two faces of one fact: A^* is generated freely from ϵ and A by appending tokens, and it is both the initial algebra for $1 + X \times A$ and the free monoid on A. Second, the reachability map r_Q of §1.3.1 is this unique algebra morphism, and Q is reachable (Definition 1.19) iff it is onto: the image of r_Q is the smallest subset of Q containing q_0 and closed under transitions.

1.4.3 Behaviours and the final coalgebra

Dually to the strings, the machines themselves carry a coalgebra structure, and it is final.

Definition 1.32 (Residuals)**.** The *residual* of a machine $f : A^* \to B$ at a string c is the machine $f_c : A^* \to B$, $f_c(w) = f(cw)$: the behaviour of f after c has been read.

The set $Z = B^{A^*}$ of all machines is itself an F-coalgebra. We write φ for a general element of Z, i.e. an arbitrary machine $\varphi : A^* \to B$, keeping f for a particular machine that is to be realised. For a token a, φ_a is the residual of φ at the one-token string a.

Theorem 1.33 (The final coalgebra)**.** *Let $Z = B^{A^*}$, the set of all machines, with coalgebra structure*

$$\zeta(\varphi) = \big(\varphi(\epsilon),\ a \mapsto \varphi_a\big),$$

so that the output of a machine is its value on the empty string and the transition on a passes to the residual. Then (Z, ζ) is the final F-coalgebra: for every F-coalgebra (Q, α) there is exactly one morphism of F-coalgebras $\mathrm{beh}_Q : Q \to Z$*, namely*

$$\mathrm{beh}_Q(q)(w) = \lambda(q \cdot w),$$

the behaviour *of the state q: the machine obtained by starting in q. In diagrams, there is a unique* beh_Q *making*

$$\begin{array}{ccc} Q & \xrightarrow{\mathrm{beh}_Q} & Z \\ {\scriptstyle \alpha}\downarrow & & \downarrow{\scriptstyle \zeta} \\ B \times Q^A & \xrightarrow[\mathrm{id}\times\mathrm{beh}_Q^A]{} & B \times Z^A \end{array}$$

commute.

Proof. Finality means two things: a morphism $Q \to Z$ exists, and it is unique. Read componentwise, a map $h : Q \to Z$ is a morphism of F-coalgebras iff, for every state q and token a,

$$h(q)(\epsilon) = \lambda(q) \quad \text{(the outputs agree)}, \qquad h(q \cdot a) = h(q)_a \quad \text{(the transitions agree)},$$

since $\zeta(h(q)) = \big(h(q)(\epsilon),\, a \mapsto h(q)_a\big)$ and $F(h)(\alpha(q)) = \big(\lambda(q),\, a \mapsto h(q \cdot a)\big)$.

Existence. beh_Q satisfies both: $\mathrm{beh}_Q(q)(\epsilon) = \lambda(q \cdot \epsilon) = \lambda(q)$, and for every string w, $\mathrm{beh}_Q(q \cdot a)(w) = \lambda(q \cdot aw) = \mathrm{beh}_Q(q)(aw) = \mathrm{beh}_Q(q)_a(w)$.

Uniqueness. Let h be any morphism. Evaluating the second condition at a string w gives $h(q)(aw) = h(q \cdot a)(w)$. So $h(q)(\epsilon) = \lambda(q)$, and the value of $h(q)$ on a string of length $n+1$ is determined by the values of h on strings of length n. By induction on the length of the string, $h(q)(w) = \lambda(q \cdot w)$ for every w, i.e. $h = \mathrm{beh}_Q$. □

Remark 1.34 (Why the set of all machines is final). Uniqueness has a simple meaning: a state's behaviour is completely pinned down by its output and by the behaviours of the states it moves to, and there is nothing else to choose. Dually to Lambek's lemma for initial algebras, the structure map $\zeta : Z \to B \times Z^A$ of a final coalgebra is a bijection. Here that says something concrete: a machine is the same thing as its value on the empty string together with one residual machine for each token, since every non-empty string begins with a unique token. Finality is the abstract form of *definition by corecursion*: to specify a machine, it suffices to say what it outputs now and what machine it becomes after each token.

Corollary 1.35 (Realisations and equivalence classes)**.**

1. *A pointed state machine (Q, q_0) realises f iff $\mathrm{beh}_Q(q_0) = f$. Two pointed state machines realise the same machine iff their initial states have the same behaviour.*
2. *Machines correspond one-to-one to the* equivalence classes *of pointed state machines under "same behaviour": the final coalgebra indexes these classes. The category $\mathsf{Real}(f)$ of realisations of f is the fibre over f of the functor sending a pointed coalgebra to the behaviour of its point.*

Proof. (1) $\lambda(q_0 \cdot c) = \mathrm{beh}_Q(q_0)(c)$, so (Q, q_0) realises f iff $\mathrm{beh}_Q(q_0) = f$. (2) restates (1). □

Remark 1.36 (Behavioural equivalence). Two states, possibly in different machines, are *behaviourally equivalent* if they have the same image in Z. For this functor behavioural equivalence coincides with *bisimilarity*, the existence of a relation between the state spaces that is respected by outputs and transitions (Rutten, 2000). "Realising the same machine" in the sense of §1.3.1 is exactly "having bisimilar initial states".

Remark 1.37 (Reframing behaviour, realisation, and observability). Three items of §1.3.1 reappear here. The behaviour beh_Q, defined directly there, is by Theorem 1.33 the *only* map from Q to the set of machines that preserves outputs and transitions. Part (1) of Corollary 1.35 is the realisation condition $\mathrm{beh}_Q(q_0) = f$; part (2), which identifies a machine with the class of its realisations, is new. And a state machine is observable (Definition 1.19) iff no two of its states are behaviourally equivalent, i.e. iff beh_Q is one-to-one. Reachability and observability are thus the two halves of one picture,

$$A^* \xrightarrow{r_Q} Q \xrightarrow{\mathrm{beh}_Q} Z,$$

the unique algebra morphism from the initial algebra followed by the unique coalgebra morphism to the final coalgebra.

Example 1.38 (Parity in the final coalgebra)**.** For the parity state machine of Example 1.20, the behaviour of even is the parity machine f itself, and the behaviour of odd is its complement $1 - f$, which is also the residual f_a. So beh_Q sends the two states to two distinct points of Z: the state machine is observable, and its states are identified with the residuals of f.

The correspondence so far:

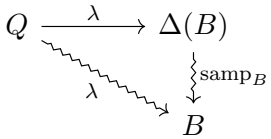
State machines (§1.3)	Coalgebras
state machine (Q, τ, λ)	F-coalgebra
state machine with initial state	pointed F-coalgebra
extension of τ to strings from q_0	unique algebra morphism from A^*
machine $f : A^* \to B$	point (element) of the final coalgebra Z
behaviour of a state	image under the unique map to Z
realisation of f	pointed coalgebra whose point has behaviour f
realisations of the same machine	pointed coalgebras over the same point of Z

1.4.4 The variants as coalgebras

The state machines of §1.3.2 have coalgebraic descriptions too. The first three need nothing beyond the functor F with a different output set, or a law on the states; random transitions need a different functor.

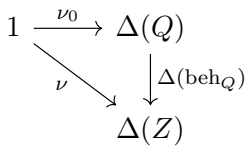
Probabilistic predictors. Replacing B by $\Delta(B)$ gives the functor $F_\Delta(Q) = \Delta(B) \times Q^A$. State machines realising probabilistic predictors are F_Δ-coalgebras, and by Theorem 1.33 the final F_Δ-coalgebra is $\Delta(B)^{A^*}$, the set of all probabilistic predictors.

Stochastic state machines. A stochastic state machine has exactly the same data as an F_Δ-coalgebra: deterministic transitions and an output map $Q \to \Delta(B)$. The difference is only in how the output is read. For a predictor the output map is a function into $\Delta(B)$; for a stochastic state machine it is a stochastic map $Q \rightsquigarrow B$, i.e. the same function followed by sampling (Proposition 1.13):



So stochastic state machines are F_Δ-coalgebras, and a stochastic state machine realises a stochastic map iff the same coalgebra realises the probabilistic predictor with the same data.

Random initial states. A state machine with random initial state is an F-coalgebra Q together with a law $\nu_0 \in \Delta(Q)$ in place of a point. Pushing it forward along the unique morphism to the final coalgebra gives a law on machines,



so a finitely supported random map is a point of $\Delta(Z)$, a distribution over points of the final coalgebra, just as a machine is a point of Z; in general it is a point of the Giry monad applied to Z.

Random transitions. A probabilistic state machine is a coalgebra for a different functor,

$$H(Q) = \Delta(B) \times \Delta(Q)^A, \qquad q \longmapsto \big(\lambda(q),\ a \mapsto \tau(q,a)\big),$$

since the transition on each token now gives a distribution over states. H-coalgebras have a final coalgebra in Set, but it identifies states up to *probabilistic bisimilarity*, which is finer than having the same output laws; §1.5.3 returns to this.

1.5 Minimal realisations

We can now ask how few states a state machine realising a given machine can have.

1.5.1 The Myhill–Nerode theorem

Definition 1.39 (Future-equivalence)**.** Strings c, c' are *future-equivalent* for f, written $c \equiv_f c'$, if $f(cw) = f(c'w)$ for every continuation $w \in A^*$.

Two strings are future-equivalent when nothing that could follow can tell them apart through the machine's outputs. The equivalence classes are the states the machine cannot avoid keeping track of. Future-equivalence identifies strings by what they imply about outputs to come, not by what they contain. Two very different strings can share a state, and two nearly identical ones can differ.

Theorem 1.40 (Minimal state machine; Myhill–Nerode)**.** *Let f be any machine.*

1. *The classes of $\equiv_f$ form a state machine realising f: states $Q_f = A^*/\!\equiv_f$, initial state $[\epsilon]$, transitions $\tau([c], a) = [ca]$ and outputs $\lambda([c]) = f(c)$.*
2. *Any state machine realising f has at least $|Q_f|$ reachable states.*
3. *If Q_f is finite, the reachable part of a realisation with exactly $|Q_f|$ reachable states is isomorphic to this one.*

In particular f is realised by a finite state machine iff $\equiv_f$ has finitely many classes (Myhill, 1957; Nerode, 1958).

Proof. (1) The maps are well defined: if $c \equiv_f c'$ then $f(c) = f(c')$ (take $w = \epsilon$), and $ca \equiv_f c'a$ (since $f(caw) = f(c'aw)$ for all w). By induction $\tau([\epsilon], c) = [c]$, so the output on c is $f(c)$.

(2) Let (Q, q_0, τ, λ) realise f and send each reachable state $q = q_0 \cdot c$ to $[c]$. This is well defined: if $q_0 \cdot c = q_0 \cdot c'$ then $q_0 \cdot cw = q_0 \cdot c'w$ for every w, so $f(cw) = f(c'w)$ and $c \equiv_f c'$. It is onto Q_f, so there are at least $|Q_f|$ reachable states.

(3) If Q_f is finite and there are exactly $|Q_f|$ reachable states, the onto map in (2) is between finite sets of the same size, so it is a bijection, and it commutes with transitions and outputs. (For infinite Q_f counting states is not enough: an onto map between infinite sets of the same size need not be one-to-one. Theorem 1.44 gives the version that holds in general.) □

Example 1.41 (State counts for Example 1.2)**.**

1. Constant: one state.

2. Last token: two states, "ends in a" and "does not" (the empty string behaves exactly like one ending in b).
3. Parity: two states, however long the string.
4. Balance: infinitely many. The classes are indexed by the difference $d(c) = \#a - \#b$: strings with different d are separated by appending a string that brings one of them back to balance.
5. Memoriser: finitely many, but growing with the data. Every string that is not a prefix of some example c_i has all continuations mapped to 0, so these form a single class; the remaining classes are at most the prefixes of the examples, about the total length of the training strings.

Remark 1.42 (Memorising versus modelling, first look). The third and fifth examples show the contrast that motivates Chapter 6. A memoriser needs a number of states that grows with its training data, and it says nothing useful about strings it has not seen. The parity machine has two states however much data it is shown, and it is correct on every string. When a machine's behaviour is generated by a small state machine, storing that state machine is cheaper than storing examples. The stochastic version of Theorem 1.40, where outputs are distributions over the next token, gives the *causal states* of Chapter 4, and the same comparison becomes one of its central questions.

1.5.2 The categorical version

In the language of §1.4, the Myhill–Nerode theorem has a universal formulation, which explains why its construction works.

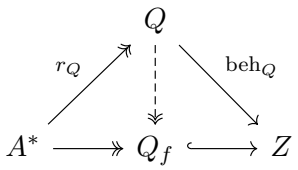
Corollary 1.43 (The canonical minimal realisation). *Each equivalence class of realisations (Corollary 1.35) has a canonical representative: the subcoalgebra of Z generated by f, whose states are the residuals $\{f_c : c \in A^*\}$, with f as initial state. It is the minimal realisation of Theorem 1.40, and $c \equiv_f c'$ iff $f_c = f_{c'}$.*

Proof. The residuals are closed under the transitions of Z, since $(f_c)_a = f_{ca}$, so they form a subcoalgebra, and it realises f from the point f. Its states are indexed by $A^*/\equiv_f$ because $f_c = f_{c'}$ is the definition of $c \equiv_f c'$. □

Theorem 1.44 (Minimal realisation as an image factorisation). *For a pointed state machine Q realising f, the unique algebra map from the initial algebra and the unique coalgebra map to the final coalgebra compose to*

$$A^* \xrightarrow{c \mapsto q_0 \cdot c} Q \xrightarrow{\mathrm{beh}_Q} Z, \qquad c \longmapsto f_c,$$

which depends only on f. By Definition 1.19 and the remarks of §1.4, Q is reachable iff the first map is onto and observable iff the second is one-to-one. Call Q minimal *if it is both. All minimal realisations are isomorphic to the image of $c \mapsto f_c$ in the epi–mono factorisation of the canonical map $A^* \to Z$, and when Q_f is finite they are exactly the realisations with the fewest states. For reachable Q:*



Every reachable realisation sits between the free realisation A^* and the minimal one Q_f, and maps onto Q_f.

Proof. The composite sends c to $\mathrm{beh}_Q(q_0 \cdot c) = f_c$. If both maps are onto and one-to-one respectively, Q is in bijection with the image. If Q is reachable, it maps onto the image, so it has at least as many states. An unreachable state can be removed, and two states with the same behaviour can be identified, without changing the behaviour of q_0. □

Remark 1.45 (The two proofs are the same proof). The elementary proof of Theorem 1.40 is the categorical proof with the final coalgebra made explicit. The quotient $A^* \twoheadrightarrow A^*/\equiv_f$ is the epi part of the factorisation, because $c \equiv_f c'$ says exactly that c and c' have the same residual, i.e. the same image in Z; and $[c] \mapsto f_c$ is the mono part. The map $q \mapsto [c]$ used in part (2) of the elementary proof is beh_Q followed by the identification of the image with $A^*/\equiv_f$, and parts (2) and (3) say that every reachable realisation maps onto the image, bijectively exactly when it is observable. What the categorical version adds is the reason the construction works: the minimal state after c is not merely an equivalence class of strings but a point of the final coalgebra, the residual f_c. This is the automata analogue, in categorical form, of Kalman's decomposition of a linear system into its reachable and observable parts, and of the realisation theory of Arbib and Manes (Arbib and Manes, 1974; Rutten, 2000); it makes "a state is defined by its future" a theorem.

1.5.3 Minimal realisations of the variants

The minimal-realisation theory above was stated for deterministic machines. For the variants of §1.2, part of it carries over at once, and part of it is genuinely harder.

Probabilistic predictors. A probabilistic predictor is a deterministic machine whose output set is $\Delta(B)$, so everything above applies with $\Delta(B)$ in place of B.

Corollary 1.46 (Minimal probabilistic predictors)**.** *Let $f : A^* \to \Delta(B)$ be a probabilistic predictor. Strings c, c' are future-equivalent iff $f(cw) = f(c'w)$ for every continuation w, the minimal realisation of f has the residuals $\{f_c : c \in A^*\}$ as states, and the final coalgebra is $\Delta(B)^{A^*}$.*

Proof. Theorem 1.40, Theorem 1.33 and Corollary 1.43, with output set $\Delta(B)$. □

Remark 1.47 (Forward to Chapter 4: next-token predictors). Nothing above requires B to be a set of tokens. Take the output set to be $\Delta(A)$, i.e. $B = A$ in Corollary 1.46, so that $f(c)$ is a distribution over the next token. Then f is a next-token predictor, a realisation of f is a state machine whose states emit next-token distributions and move deterministically on each observed token (a *unifilar* hidden Markov model), and its minimal realisation has as states the classes of strings with the same residual predictor. When every $f(c)$ has full support, as it does for a softmax network, two strings have the same residual iff they assign the same probabilities to every future continuation. These are the *causal states* of Chapter 4, restricted to finite strings: the same construction, applied to a machine with outputs in the simplex.

Stochastic maps. For stochastic state machines (Definition 1.22), minimality reduces to the case of predictors.

Corollary 1.48 (Minimal stochastic state machines)**.** *A stochastic state machine realises the stochastic map f iff, read as a deterministic machine with outputs in $\Delta(B)$, it realises the probabilistic predictor with the same data. The minimal stochastic state machine realising f is therefore the minimal realisation of Corollary 1.46.*

Proof. Both conditions say $\lambda(q_0 \cdot c) = f(c)$ for every c. □

Random maps. For state machines with random initial state (Definition 1.23), the minimal realisation is generated by the machines that can be drawn.

Corollary 1.49 (Minimal realisation of a random map)**.** *Let ν be the law of a random map, finitely supported, and let $\langle \operatorname{supp} \nu \rangle \subseteq Z$ be the subcoalgebra of the final coalgebra generated by the machines in the support of ν, whose states are the residuals $\{\varphi_c : \varphi \in \operatorname{supp} \nu,\ c \in A^*\}$. With initial law ν it realises ν, and every reachable realisation maps onto it; when it is finite, it therefore has the fewest states.*

Proof. With initial law ν on $\langle \operatorname{supp} \nu \rangle$, the behaviour of a state φ is φ itself, so the behaviour of the random initial state has law ν. For any realisation, beh_Q is a coalgebra morphism into Z that sends the support of ν_0 onto the support of ν; it therefore sends the reachable part of Q onto $\langle \operatorname{supp} \nu \rangle$. □

Example 1.50 (The two coins again)**.** In Example 1.5, F_1 is the constant machine 0 or the constant machine 1, each with probability $\frac{1}{2}$. Its minimal realisation has two states, each sending every token back to itself, one with output 0 and one with output 1, and the initial state is chosen by a fair coin. By contrast, the stochastic map with $f(c) = \frac{1}{2}\delta_0 + \frac{1}{2}\delta_1$ is realised by a stochastic state machine with a *single* state that emits a fair coin at every step. That stochastic map behaves like F_2, whose law is a product measure on B^{A^*} giving every individual machine probability zero; as a random map, F_2 is not finitely supported and has no finite realisation of this kind. The comparison shows how differently the two kinds of randomness are stored: F_1 stores one fair bit forever, while a stochastic map draws a fresh bit at every step and stores nothing.

Random transitions: the hard case. The remaining case is a probabilistic state machine (Definition 1.24), whose *transitions* are random, so that the state after a string is not determined by the string. Such machines are hidden Markov models, and their minimal realisations are the subject of Chapter 5. Three things change.

Remark 1.51 (Why random transitions are harder).

1. *The state is not a function of the string.* An observer knows only a distribution over hidden states, updated by Bayes' rule after each token. The analogue of future-equivalence must be formulated for these belief states, which needs the updating machinery of Chapter 2.
2. *Minimality is not a quotient.* For deterministic updates the minimal realisation is a quotient of every reachable one. With random transitions, the natural minimal object is the minimal *linear* dimension, the rank of a Hankel matrix (Carlyle, 1963; Schützenberger, 1961); the minimal number of *states* is the much harder positive-realisation problem (Chapter 5), and has no quotient construction.

3. *The final coalgebra changes meaning.* For functors with random transitions, such as H of §1.4.4, the final coalgebra in Set identifies states up to *probabilistic bisimilarity* (Larsen and Skou, 1991), which is strictly finer than having the same law of outputs for every input sequence. The latter, trace equivalence, is captured by final coalgebras in a Kleisli category instead (Hasuo et al., 2007).

Machines and their realisations are now in place. A next-token predictor is used repeatedly, its outputs feeding its inputs; what it must output for that to make sense, and what its repeated use determines, is the subject of the next chapter.

Bibliography

Michael A. Arbib and Ernest G. Manes. Machines in a category: An expository introduction. *SIAM Review*, 16:163–192, 1974.

J. W. Carlyle. Reduced forms for stochastic sequential machines. *Journal of Mathematical Analysis and Applications*, 7:167–175, 1963.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, et al. Toy models of superposition. Transformer Circuits Thread; arXiv:2209.10652, 2022.

Michèle Giry. A categorical approach to probability theory. In *Categorical Aspects of Topology and Analysis*, volume 915 of *Lecture Notes in Mathematics*, pages 68–85. Springer, 1982.

Ichiro Hasuo, Bart Jacobs, and Ana Sokolova. Generic trace semantics via coinduction. *Logical Methods in Computer Science*, 3(4), 2007.

Jesse Hoogland, George Wang, Matthew Farrugia-Roberts, Liam Carroll, Susan Wei, and Daniel Murfet. Loss landscape degeneracy drives stagewise development in transformers. *Transactions on Machine Learning Research*, 2025. arXiv:2402.02364.

Evan Hubinger, Chris van Merwijk, Vladimir Mikulik, Joar Skalse, and Scott Garrabrant. Risks from learned optimization in advanced machine learning systems. arXiv:1906.01820, 2019.

Jared Kaplan, Sam McCandlish, Tom Henighan, et al. Scaling laws for neural language models. arXiv:2001.08361, 2020.

Joachim Lambek. A fixpoint theorem for complete categories. *Mathematische Zeitschrift*, 103: 151–161, 1968.

Lauro Langosco, Jack Koch, Lee Sharkey, Jacob Pfau, and David Krueger. Goal misgeneralization in deep reinforcement learning. In *Proceedings of the 39th International Conference on Machine Learning*, 2022.

Kim G. Larsen and Arne Skou. Bisimulation through probabilistic testing. *Information and Computation*, 94:1–28, 1991.

George H. Mealy. A method for synthesizing sequential circuits. *Bell System Technical Journal*, 34:1045–1079, 1955.

Edward F. Moore. Gedanken-experiments on sequential machines. In C. E. Shannon and J. McCarthy, editors, *Automata Studies*, pages 129–153. Princeton University Press, 1956.

John Myhill. Finite automata and the representation of events. Technical Report WADC Technical Report 57-624, Wright Air Development Center, 1957.

Anil Nerode. Linear automaton transformations. *Proceedings of the American Mathematical Society*, 9:541–544, 1958.

Catherine Olsson, Nelson Elhage, Neel Nanda, et al. In-context learning and induction heads. Transformer Circuits Thread; arXiv:2209.11895, 2022.

Michael O. Rabin and Dana Scott. Finite automata and their decision problems. *IBM Journal of Research and Development*, 3:114–125, 1959.

Daniel A. Roberts, Sho Yaida, and Boris Hanin. *The Principles of Deep Learning Theory*. Cambridge University Press, 2022. arXiv:2106.10165.

J. J. M. M. Rutten. Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249:3–80, 2000.

Marcel-Paul Schützenberger. On the definition of a family of automata. *Information and Control*, 4:245–270, 1961.

Rohin Shah, Vikrant Varma, Ramana Kumar, Mary Phuong, Victoria Krakovna, Jonathan Uesato, and Zac Kenton. Goal misgeneralization: Why correct specifications aren't enough for correct goals. arXiv:2210.01790, 2022.

Adam S. Shai, Sarah E. Marzen, Lucas Teixeira, Alexander Gietelink Oldenziel, and Paul M. Riechers. Transformers represent belief state geometry in their residual stream. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2405.15943.

Adly Templeton, Tom Conerly, Jonathan Marcus, et al. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. Transformer Circuits Thread, 2024.

Sumio Watanabe. *Algebraic Geometry and Statistical Learning Theory*. Cambridge University Press, 2009.